



World Robot Olympiad (WRO): FUTURE ENGINEERS

Jawatha Fighters

- Introduction
- Our Team
- Robot Design
- Design and Prototype
- Steering System
- 3D Modeling
- Robot capabilities
- Camera
- Coding
- Code explained

Introduction

This project details the development of Sensoria, an autonomous robot engineered for the WRO Future Engineers challenge. Our team combined skills in C++ programming, 3D CAD modeling, and circuit design to create a custom vehicle capable of completing multi-lap track navigation and obstacle avoidance. Designed around strict size limits, low center-of-gravity requirements, and real-time vision processing constraints, Sensoria autonomously detects color-coded traffic blocks and uses sensor feedback to make split-second steering corrections while maintaining high-speed lap stability.

Our Team

The team comprises of an expert programmer, a skilled 3D modeler, and a experienced coach. Our combined expertise ensures high-quality results.



Hasan Abo Fares

Coach:

He is responsible for construction guidance, overall project organization, team building, and strategy coordination.



Mohammed Almossa

Lead Software
Engineer:

Responsible for developing the core autonomous algorithms, sensor integration, and primary control loops.



Mohammed Alsalman

Lead Mechanical &
Electrical Engineer:

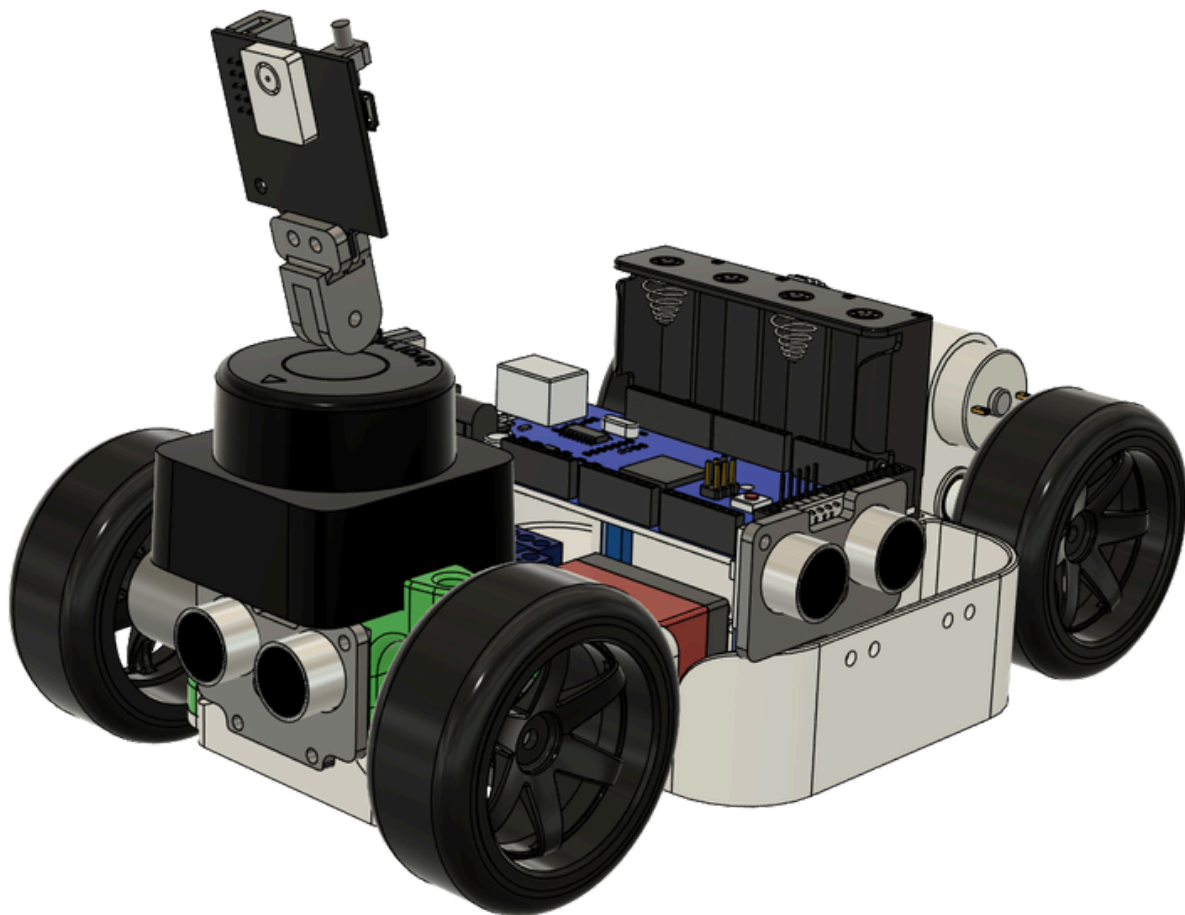
Responsible for designing the custom robot chassis in CAD, structural 3D modeling, and managing the circuit wiring layout.

Robot Design

Sensoria's Base

Sensoria utilizes a 100% custom 3D-printed chassis rather than an off-the-shelf structural frame. A custom robot base enhances structural strength by optimizing material distribution specifically for our competition layout, directly improving overall stability, durability, and power routing. Designing our own chassis also allowed us to tailor specific mounting features and cable channels to our exact dimensions, making the vehicle far better suited for the high-speed maneuvers of the WRO track compared to mass-produced bases.

- Dimensions: 210 mm (L) × 145 mm (W) × 110 mm (H)
- Primary Material: PLA+ for high impact resistance and minimal warping.

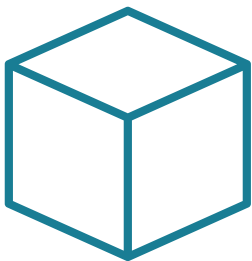


Robot Design

Sensoria's Form

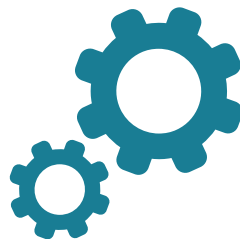
Sensoria features a custom chassis entirely modeled in Autodesk Fusion 360 and brought to life through 3D printing. Moving beyond pre-fabricated frames, standard 3D modeling enabled us to tailor every millimeter of the physical frame to meet the specific demands of the WRO Future Engineers track.

Observed Benefits of Our Fully 3D-Modeled Design:



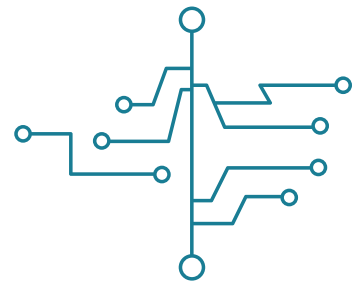
Precision Component Integration:

Modeling every sensor, servo, and board in Fusion 360 allowed us to verify exact physical clearances before printing. This eliminated mounting errors and ensured our 5 ultrasonic sensors and Pixy2 camera sat at the exact angles needed for accurate arena tracking.



Rapid Physical Iteration:

When testing revealed chassis flex near the steering assembly, we directly modified the CAD model, added structural support gussets, and printed a revised frame version in hours. This drastically reduced setup time and development costs compared to traditional machining.



Streamlined Internal Cabling:

We integrated custom cable channels and isolation barriers directly into the floor pan model. This clean routing prevented signal interference between motor wires and sensor buses while keeping the overall layout compact.

Robot Design Prototyping

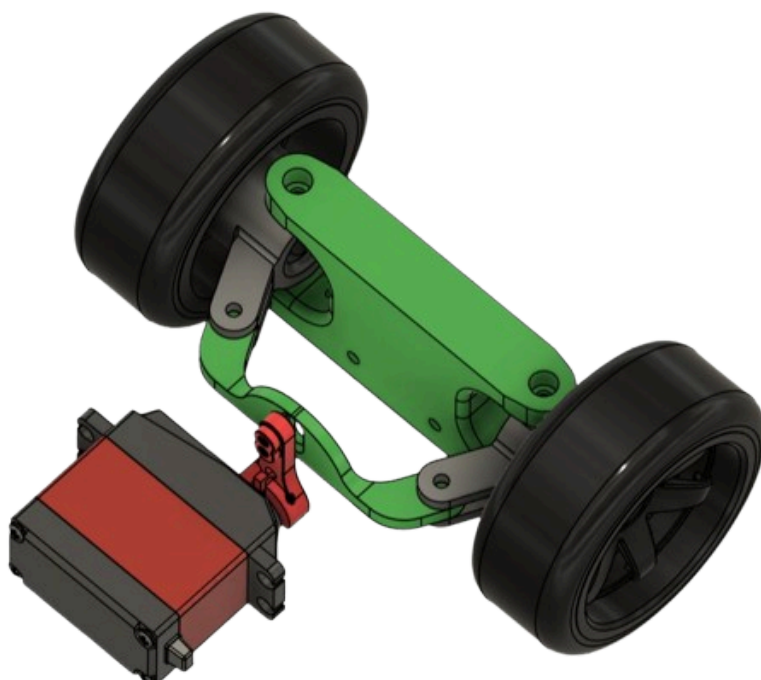
Sensoria evolved through multiple physical 3D-printed prototypes to isolate vulnerabilities, perform track validation, and iteratively refine our vehicle layout. Fabrication and stress testing of every physical revision ensured our final iteration met all WRO dimensional limits and performance goals.

Iteration :	Identified Issue	Mechanical Change Applied	Measured Test Result
Prototype v1.0	High frame flex near front steering hub during tight cornering.	Added 2.5 mm vertical support ribs and gussets to the front subframe.	Frame deflection reduced by ~70%; steering response became crisp and predictable.
Prototype v2.0	Side ultrasonic sensors suffered blind spots when approaching wall corners at steep angles.	Angled outer sensor mounts outward by 15° relative to the longitudinal axis	Eliminated wall detection blind spots and reduced false distance spikes.
Prototype v3.0	Rapid acceleration caused camera tower vibration, throwing off Pixy2 color tracking.	Integrated a dual-braced elevated camera mount directly into the main chassis floor pan.	Stabilized vision feed, maintaining clean block tracking at maximum drive speeds

Robot Steering System

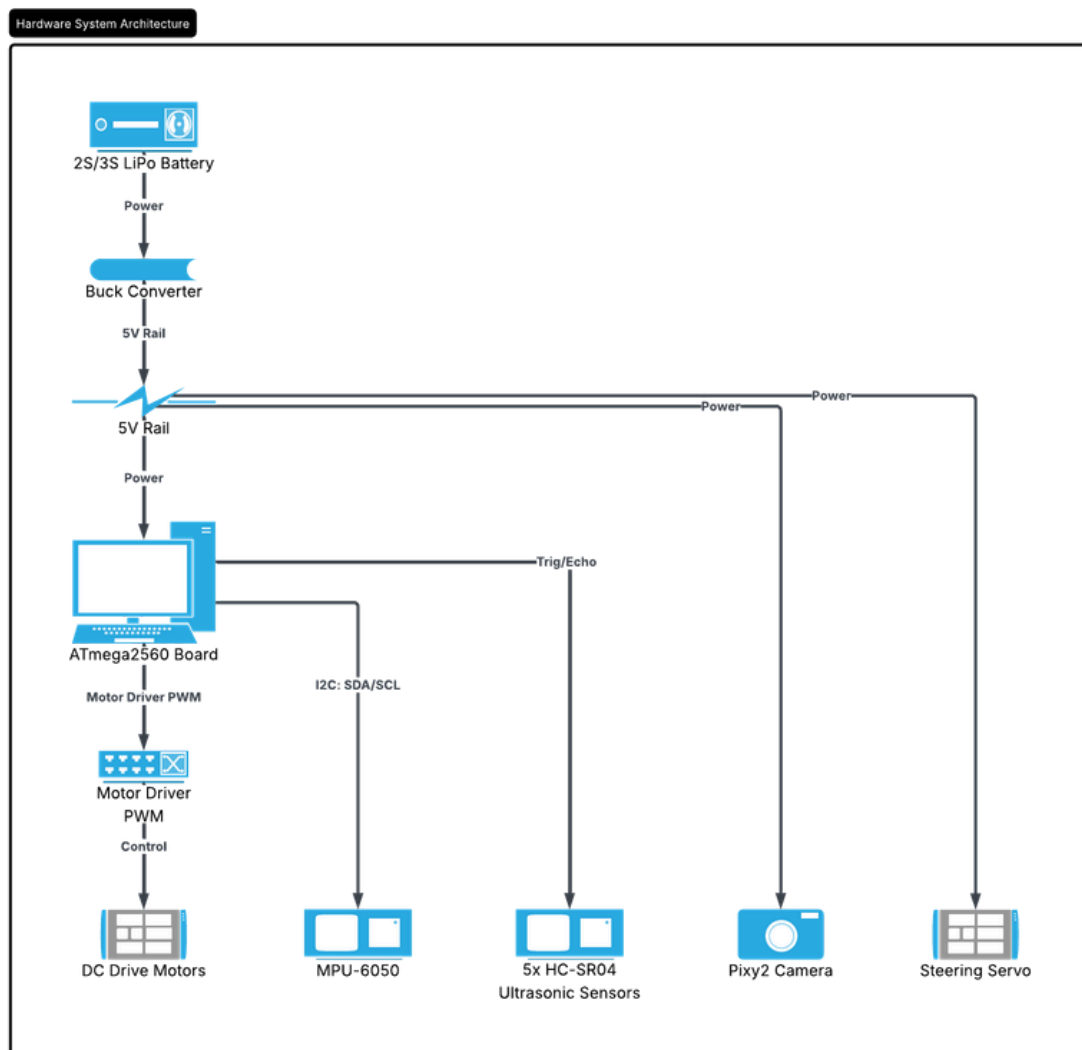
Sensoria utilizes an Ackerman-inspired steering geometry powered by a high-torque 20kg digital servo motor.

- Torque Rating: 20 kg-cm at 6.0V.
- Range of Motion: Software-constrained between 65° and 115° (center at 90°) to prevent mechanical binding.
- Mounting & Linkage: Reinforced tie-rods with ball joints connect directly to custom 3D-printed steering knuckles.



Electrical Architecture & Microcontroller integration.

The processing system relies on an ATmega2560-based microcontroller board (Arduino Mega layout) running at 16 MHz with 256 KB Flash Memory.



Computer Vision: Pixy2 Setup & Data Processing

The primary optical sensing system on Sensoria is the Pixy2 CMUcam5 smart vision sensor mounted on an elevated front tower. Offloading image processing entirely to the Pixy2's dedicated dual-core NXP LPC4330 processor (running at 60 FPS) frees up the main ATmega2560 microcontroller to run low-latency steering control loops.

Sensor	Quantity	Function & Placement	Operational Purpose
Pixy2 Camera	1	Forward-center, elevated tower.	Detects color signatures of obstacle blocks (red/green).
HC-SR04 Ultrasonic	5	Arrayed around front and side bumper positions.	Wall distancing, lane retention, and proximity checks.
MPU-6050 (IMU)	1	Centered on chassis floor (near center of gravity).	Yaw tracking, heading correction, and straight-line verification.

The primary optical sensing system on Sensoria is the **Pixy2 CMUcam5 smart vision sensor** mounted on an elevated front tower. Offloading image processing entirely to the Pixy2’s dedicated dual-core NXP LPC4330 processor (running at 60 FPS) frees up the main ATmega2560 microcontroller to run low-latency steering control loops.

Color Connected Components (CCC) Algorithm

The camera detects track obstacle blocks using a hardware-accelerated Color Connected Components (CCC) algorithm:

Hue & Saturation Color Space: Unlike standard RGB processing (which shifts heavily under changing light levels), the CCC algorithm isolates the Hue and Saturation of pixels. This renders target recognition resilient to natural shadows and ambient lighting changes on the track.

Parameter	Operational Value	Technical Rationale
Field of View (FOV)	60° Horizontal × 40° Vertical	Provides early detection of distant obstacle blocks while maintaining sufficient side margins.
Camera Frame Rate	60 Frames Per Second (FPS)	Minimizes motion blur during high-speed forward motion.
Brightness & Exposure	Fixed (Manual Exposure Lock)	Automatic Gain/Exposure is disabled in software to eliminate dynamic white balance drift.
Min/Max Block Size	min_block_area = 150 pixels	Filters out high-frequency optical noise, small reflections, and background visual artifacts.

MPU-6050 Gyro & Yaw Processing

Sensor Calibration & Zero-Motion Drift Offsets

- **Startup Self-Calibration Routine:** Upon power-up, the robot remains stationary for two seconds while the controller gathers baseline samples to set initial gyroscope zero-offsets.
- **Gyroscope Drift Compensation:** Standard micro-electro-mechanical gyroscopes suffer from temporal integration drift over extended run times. The DMP fuses accelerometer gravity measurements with raw angular velocity to stabilize yaw readings and prevent cumulative orientation errors over multi-lap runs.

•

Closed-Loop Directional Control Logic

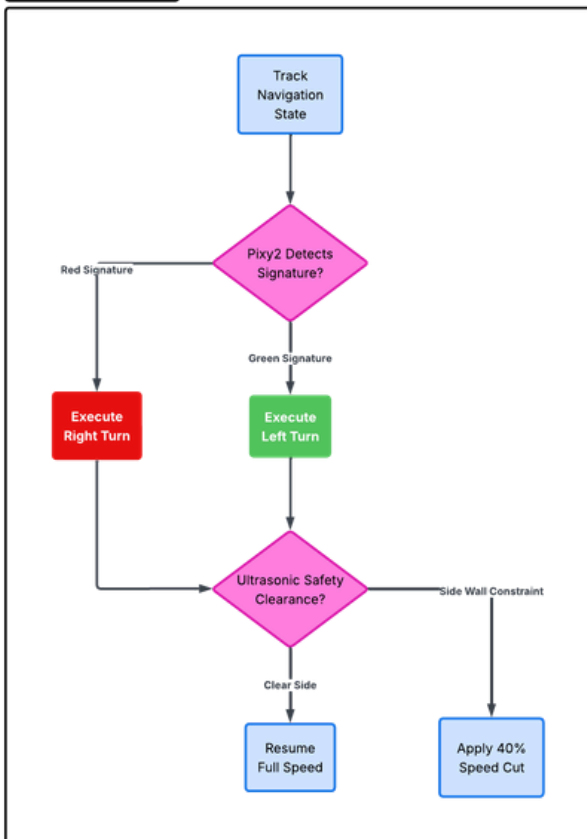
To achieve reliable straight-line driving and prevent uneven motor traction from pulling the vehicle off course, the software implements a closed-loop heading correction system:

- **Target Angle Reference:** Upon entering a straight track segment or completing an avoidance maneuver, the controller locks the target heading angle.
- **Yaw Angle Error Calculation:** Real-time orientation read over the I2C bus is continuously subtracted from the target angle to calculate angular deviation
- **Proportional Steering Adjustment:** The calculated heading deviation is multiplied by a tuned gain coefficient to determine the exact degree shift applied to the 20kg digital steering servo. If the vehicle drifts left, the controller applies proportional right-steering correction to realign the chassis axis with the target direction.

Obstacle Avoidance Strategy & Edge Cases

The obstacle avoidance architecture uses hierarchical sensor fusion to combine forward vision data from the Pixy2 camera with proximity distance readings from the 5-point HC-SR04 ultrasonic array.

Obstacle Avoidance State Machine



Primary Avoidance Decision Matrix

Obstacle navigation is determined by color-coded traffic blocks detected in the camera's field of view:

- **Red Obstacle Block (Signature 1):** Indicates a right-side pass condition. The vehicle initiates a steering shift to the right of the obstacle while monitoring the front-right ultrasonic sensor to ensure adequate wall clearance.
- **Green Obstacle Block (Signature 2):** Indicates a left-side pass condition. The vehicle initiates a steering shift to the left of the obstacle while monitoring the front-left ultrasonic sensor.

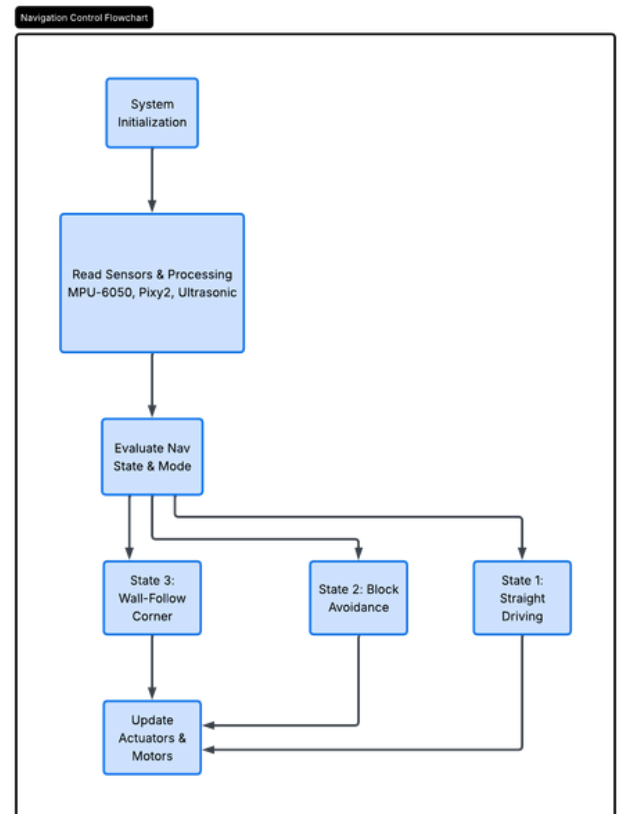
- **Bounding Box Offset Tracking:** The lateral position of the block bounding box within the camera image frame dictates steering aggressiveness. As the target block moves closer to the vehicle frame, steering angles increase dynamically to ensure clear structural clearance.

Main Control Loop Architecture & State Machine

The top-level software architecture operates as a non-blocking, deterministic finite state machine (FSM). To maintain high refresh rates across sensor inputs, long hardware delays (such as `delay()`) are strictly prohibited in the main loop execution.

Primary Operational States

- **State 1: Straight-Line Tracking:** When no obstacles are detected and side walls are parallel, the robot relies on MPU-6050 yaw feedback to correct steering drift and drive parallel to track boundaries.
- **State 2: Block Avoidance Override:** Triggered immediately when the Pixy2 camera registers Signature 1 (Red) or Signature 2 (Green). High-priority steering commands take precedent over standard wall-following logic until the block passes behind the front bumper.
- **State 3: Wall-Follow Cornering:** Triggered when the front-center ultrasonic sensor measures a wall distance under 40 cm while no obstacle blocks are detected. The steering servo swings to the target turn angle while side sensors maintain distance from the outer wall curve.



3D Modeling journey



Software

Designing our robot using Fusion 360 was an incredible journey that combined creativity, engineering, and a lot of trial and error. Fusion 360 allowed us to create detailed 3D models, simulate movements, and visualize our design before building anything physical. This software was essential for prototyping and refining our ideas.

Technology

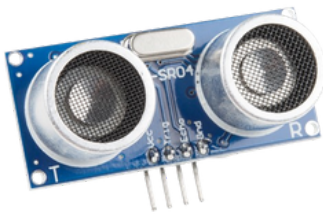
To bring our robot design to life, we relied heavily on advanced 3D printing technology and high-quality printers. The complexity of our design required precision and durability that standard printers simply couldn't provide. We needed a printer capable of handling various materials, such as strong plastics and flexible components, to accurately replicate our Fusion 360 models. The advanced 3D printer allowed us to create intricate geometries and tight tolerances that were crucial for our robot's performance.



Robot Capabilities

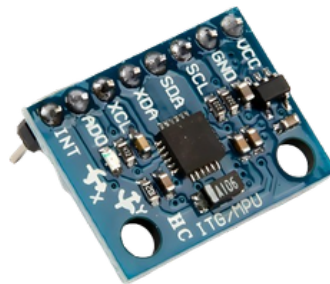
Sensoria is equipped with an array of sensors for comprehensive environmental understanding.

Sensors we used:



Ultrasonic Sensing

Sensoria uses five ultrasonic sensors positioned precisely in a designated area to detect objects in its path.



IMU Integration

The MPU6050 is important since it offers information on orientation and acceleration.

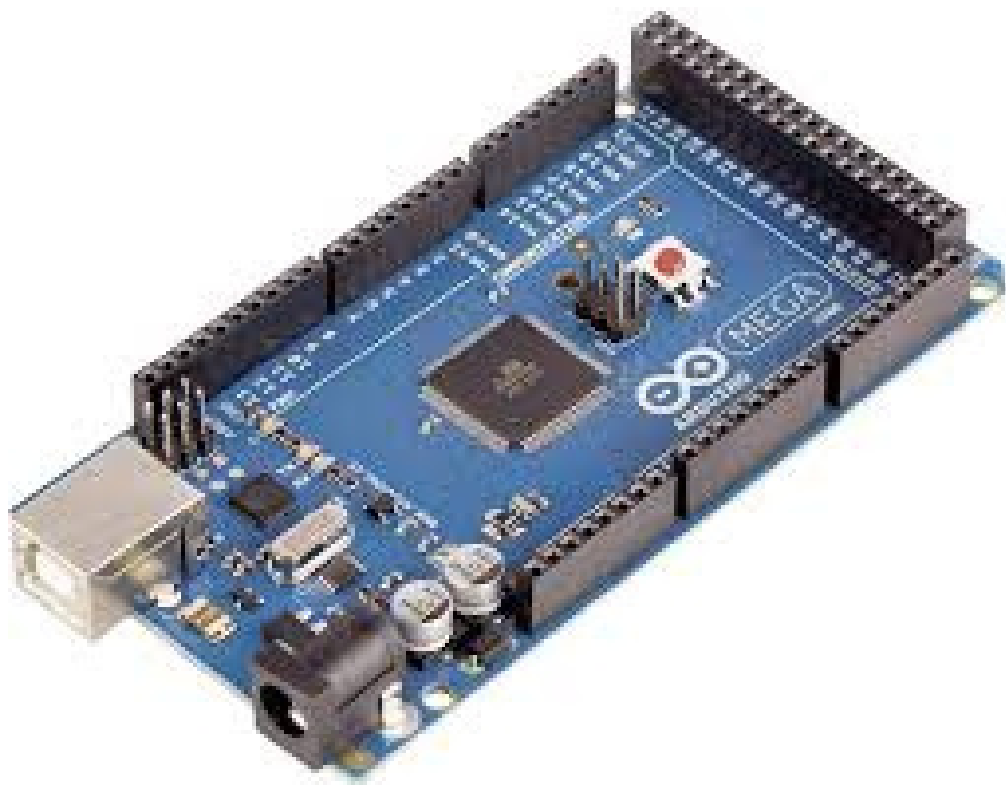


LiDAR Vision

Sensoria utilizes LiDAR for accurate 3D mapping of its surroundings.

Robot Capabilities

The Sensoria Micro-Controller, based on the ATmega2560, offers 54 digital I/O pins, 16 analog inputs, 4 UARTs, and 256 KB of flash memory. It's ideal for complex projects like robotics and home automation, supporting multiple shields and sensors with a 16 MHz clock and flexible power options.



The PIXY Camera 2

what are the reasons the camera is our main sensor

1. Light Sensitivity : Cameras can capture a wide range of light intensities, allowing them to function in various lighting conditions, from bright daylight to dimly lit environments.
2. Dynamic Range : They can record details in both highlights and shadows, providing a more complete representation of a scene compared to the human eye.
3. Color Accuracy : Cameras can reproduce a wide spectrum of colors, helping to capture the richness and vibrancy of a scene.



How we coded the robot

We utilized Arduino IDE and Pixymon, with their cutting-edge technology, to program the robot efficiently and effectively. The user-friendly interface of the IDE and Pixymon allowed us to easily write and upload code to the robot's microcontroller. The advanced features of the IDE, such as the serial monitor and debugging tools, enabled us to troubleshoot and fine-tune the robot's behavior. Overall, the Arduino IDE played a crucial role in the successful programming of the robot, showcasing the power and versatility of modern technology in robotics.



Arduino IDE



OUR CODE

Using functions in the Arduino IDE significantly improves both the efficiency and reliability of your code. By encapsulating specific tasks into functions, you make the code more modular, which reduces repetition and makes debugging easier. Functions allow you to organize your code logically, breaking it down into manageable, reusable blocks. This not only shortens the code but also makes it easier to maintain and scale for more complex projects.

The uniqueness of the code comes from how we design custom functions tailored to our specific project requirements. Every system or project has its unique parameters, inputs, and outputs, so structuring functions around these specifics makes the code fully individualized. Instead of relying on generic approaches, we create functions that directly address our goals, making the code more efficient for our use case.

Reliability is enhanced by simplifying the main loop and isolating potential sources of error within specific functions. When each function handles only one aspect of the project, it's easier to test and ensure its correct performance, thus reducing bugs or unexpected behavior. This method of coding increases the overall robustness of the system.

CODE EXPLAINED

The camera code

Using a camera to extract color signatures for robot navigation is an effective technique often employed in robotics for tasks like line following or object tracking.

The basic idea is to use the camera as a sensor to detect specific colors or patterns in the environment, which then will guide the robot's movements.

CODE EXPLAINED

The steering system code

In the steering system code, we used the `HIGH` and `LOW` commands to control the direction of the motors responsible for steering. These commands allow us to directly control the output signals to the servo, which in turn controls the movement of the steering mechanism.

```
void setup() {  
  Serial.begin(9600);  
  srituhobby.begin();  
  srituhobby.setPWMFreq(60);  
}  
  
void loop() {  
  for (int servo = 0; servo < 4; servo++ ) {  
    srituhobby.setPWM(servo, 0, servoMIN);  
    Serial.println(servo);  
    delay(300);  
  }  
  
  for (int servo = 2; servo >= 0; servo-- ) {  
    srituhobby.setPWM(servo, 0, servoMAX);  
    Serial.println(servo);  
    delay(300);  
  }  
}
```

CODE EXPLAINED

The MPU-6050 code

In the steering system code, we used the `HIGH` and `LOW` commands to control the direction of the motors responsible for steering. These commands allow us to directly control the output signals to the servo, which in turn controls the movement of the steering mechanism.

```
// MPU control/status vars
bool dmpReady = false; // set true if DMP init was successful
uint8_t mpuIntStatus; // holds actual interrupt status byte from MPU
uint8_t devStatus; // return status after each device operation (0 = success, !=0 = error)
uint16_t packetSize; // expected DMP packet size (default is 42 bytes)
uint16_t fifoCount; // count of all bytes currently in FIFO
uint8_t fifoBuffer[64]; // FIFO storage buffer

// orientation/motion vars
Quaternion q; // [w, x, y, z] quaternion container
VectorInt16 aa; // [x, y, z] accel sensor measurements
VectorInt16 aaReal; // [x, y, z] gravity-free accel sensor measurements
VectorInt16 aaWorld; // [x, y, z] world-frame accel sensor measurements
VectorFloat gravity; // [x, y, z] gravity vector
float euler[3]; // [psi, theta, phi] Euler angle container
float ypr[3]; // [yaw, pitch, roll] yaw/pitch/roll container and gravity vector

// packet structure for InvenSense teapot demo
uint8_t teapotPacket[14] = { '$', 0x02, 0,0, 0,0, 0,0, 0,0, 0x00, 0x00, '\r', '\n' };

// =====
// === INTERRUPT DETECTION ROUTINE ===
// =====

volatile bool mpuInterrupt = false; // indicates whether MPU interrupt pin has gone high
void dmpDataReady() {
```